

Reviewer Pack — Plethysm Coefficient Gap in Symmetric Powers of Permanent Po...

Entry 66a7f7df2a8b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 06:49:40 UTC

Plethysm Coefficient Gap in Symmetric Powers of Permanent Polynomials

Entry ID: 66a7f7df2a8b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 06:49:40 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Complexity of Permanent

Statement:

For any $n \geq 2$, the plethysm coefficient $c_{\lambda}^{\{\text{Sym}^k(\text{Sym}^{n(\mathbb{C}^m)})\}}$ of the permanent polynomial Perm_n satisfies $c_{\lambda} \geq 2^{\Omega(n)}$ when λ is a hook partition with arm length $\geq k$, while for the determinant Det_m , $c_{\lambda} \leq \text{poly}(m)$ for all $k \leq m^{\{1/2\}}$ and $m < n^{\{1.5\}}$.

Rationale (proposer’s reasoning):

Schur-Weyl duality decomposes tensor powers into irreducible representations, and plethysm coefficients measure how symmetric powers interact with these decompositions. Permanent polynomials exhibit exponential growth in specific hook partitions, while determinants remain polynomially bounded, reflecting their distinct orbit closure structures under $GL(n)$.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 247c1fc349f1f0d4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k, n = len(A), len(B), len(B[0])
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    Augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
                max_row = j
        Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
        for j in range(m):
            if i != j:
                factor = Augmented[j][i] / Augmented[i][i]
                for k in range(n+1):
                    Augmented[j][k] -= factor * Augmented[i][k]
    X = [0] * n
    for i in range(n-1, -1, -1):
        X[i] = Augmented[i][-1]
        for j in range(i+1, n):
            X[i] -= Augmented[i][j] * X[j]
```

```

        X[i] -= Augmented[i][j] * X[j]
    X[i] /= Augmented[i][i]
    return X

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = math.isqrt(n**1.5)

    def perm_polynomial(x):
        return sum(x[i]**n for i in range(n))

    def det_polynomial(x):
        if n == 1:
            return x[0]
        elif n == 2:
            return x[0]*x[3] - x[1]*x[2]
        else:
            det = 0
            for j in range(n):
                submatrix = [row[:j] + row[j+1:] for row in x[1:]]
                det += (-1)**j * x[0][j] * det_polynomial(submatrix)
            return det

    def plethysm_coefficient(poly, hook_partition):
        arm_length = hook_partition[0]
        if arm_length == 0:
            return 1
        coeff = 0
        for i in range(arm_length + 1):
            coeff += poly(i) * plethysm_coefficient(poly, (i,) + hook_partition[1:])
        return coeff

    perm_ratio = max(plethysm_coefficient(perm_polynomial, (k, 1**(n-k))) for k in range(n))
    det_ratio = max(plethysm_coefficient(det_polynomial, (k, 1**(m-k))) for k in range(m+1))

    return {
        "metric_name": "Ratio",
        "metric_value": perm_ratio / det_ratio,
        "instances_tested": 1,
        "conjecture_holds": perm_ratio >= 2**(n/10) and det_ratio <= n**3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = "mapping_undefined"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_432a7123.py", line 103, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_432a7123.py", line 60, in run_trial
    m = math.isqrt(n**1.5)
          ~~~~~
TypeError: 'float' object cannot be interpreted as an integer

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error; unable to evaluate conjecture validity | next: Fix integer conversion in test code (replace `math.isqrt(n1.5)` with `math.isqrt(int(n1.5))`)

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	32951
2	preregistration	ollama_remote	qwen3:8b	0	0	20028
3	novelty	ollama_remote	qwen3:8b	0	0	16565
4	novelty	ollama_remote	qwen3:8b	0	0	12329
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16942
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11893
7	judge	ollama_remote	qwen3:8b	0	0	9130

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119838 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/66a7f7df2a8b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/66a7f7df2a8b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/66a7f7df2a8b.tar.gz (if generated)